Right Resource Fit

Description

Hire Helper is an intuitive web-based platform designed to connect individuals with skilled service providers for completing everyday tasks efficiently, enabling users to post tasks such as home repairs, cleaning, moving assistance, and other services while allowing service providers to discover and accept tasks based on their skills and availability; the platform provides a seamless environment where users can easily find reliable helpers, manage their requests, and track task progress, while service providers can showcase their abilities and gain job opportunities, and by incorporating features such as OTP-based secure authentication, real-time task tracking, request management with accept/reject options, user profile management, notification system, and interactive dashboards, Hire Helper ensures a smooth, secure, and user-friendly experience across devices, ultimately reducing the time and effort involved in traditional hiring methods and offering a fast, efficient, and reliable solution for both users and workers.

Purpose

The purpose of **Hire Helper** is to streamline and modernize the process of finding and hiring local service providers by:

- **Simplifying Task Hiring:** Enable users to quickly post tasks and connect with available workers without manual searching
- **Providing Opportunities:** Help skilled workers find jobs and earn by offering their services
- **Ensuring Security:** Use OTP-based authentication for safe and secure user access
- **Improving Efficiency:** Reduce time and effort in managing tasks, requests, and communication
- **Enhancing User Experience:** Provide a clean, easy-to-use platform for both users and service providers
- **Encouraging Local Connectivity:** Build a community where people can help each other with daily needs

Getting Started

Prerequisites

Before setting up and running the application, ensure the following software and tools are installed on

your system:

- Node.js and npm

  Download and install from Node.js official website.

  npm comes bundled with Node.js

- Mongo DB

  Install and set up MongoDB locally or use a cloud-hosted solution like MongoDB Atlas.

- SMTP Service

  Configure an SMTP service (e.g., Gmail SMTP) for sending OTPs and notifications.

- IDE

  Use a development environment like Visual Studio Code for code editing and debugging.

Installation

Follow these steps to set up the project:

- **Clone the Repository**

  git clone https://github.com/springboardmentor198/RightResourceFit_Infosys_Internship_Feb2026-Team_01.git

  cd right-resource-fit

- **Install Dependencies**

  npm instal

Running the Application

- Start the Backend

  Navigate to the backend directory:

  cd backend

  Run the backend server(ensure the backend runs on a specific port, e.g.,5000):

  nodemon derver.js

- Start the Frontend

  Navigate to the frontend directory:

  cd frontend

  Run the development server:

  npm run dev

Accessing the Application

- Frontend: Visit https://localhost:5173
- Backend API: Accessible at https://localhost:5000 for testing endpoint.

Environment Setup for Right Resource Fit

To run the project seamlessly, you need to set up environment variables for both the backend and frontend. Create a .env file in the root directory of the backend project and add the required configurations as shown below.

Required Environment Variables

Variable Name	Description	Example Value
MONGO_URI	Connection string for the main database.	mongodb://localhost:27017/hirehelper
USER_DB_URL	Connection string for user database.	mongodb://localhost:27017/users
TASK_DB_URL	Connection string for tasks database.	mongodb://localhost:27017/tasks
REQUEST_DB_URL	Connection string for requests database.	mongodb://localhost:27017/requests
NOTIFICATION_DB_URL	Connection string for notifications database.	mongodb://localhost:27017/notifications
JWT_SECRET	Secret key for signing JWT tokens.	supersecretkey123
SMTP_HOST	SMTP host for sending emails (OTP).	smtp.gmail.com
SMTP_PORT	SMTP port for email service.	587
SMTP_USER	Email address used to send OTP emails.	your-email@example.com
SMTP_PASS	Password or app password for email.	your-email-password
CLIENT_URL	Frontend URL for CORS configuration.	http://localhost:3000

Example. env Configuration

Here's an example .env file for your project:

MongoDB URLs

USER_DB_URL=mongodb://localhost:27017/user

JOB_DB_URL=mongodb://localhost:27017/jobs

NOTIFICATION_DB_URL=mongodb://localhost:27017/notifications

APPLICATION_DB_URL=mongodb://localhost:27017/applicatuons

JWT Secret

JWT_SECRET=supersecretkey123

SMTP Configuration

SMTP_HOST=smtp.gmail.com

SMTP_PORT=587

SMTP_USER=your-email@example.com

SMTP_PASS=your-email-password

Steps to set up the Environment

- Create a .env file in the root of your backend directory.
- Copy the example configuration above and paste it into the .env file.
- Replace placeholder values (like your-email@example.com and your-email-password) with your actual configuration.

Additional Notes

- For cloud-hosted MongoDB (e.g., MongoDB Atlas), replace mongodb://localhost:27017/<db> with your cluster's connection string
- If you're using a third-party email service, adjust the SMTP_HOST, SMTP_PORT, SMTP_USER, and SMTP_PASS accordingly.
- Keep the .env file secure and do not commit it to version control. Add .env to your .gitignore file

Folder Structure

A well-organized folder structure helps maintain clarity and efficiency in development. Below is a suggested folder structure for your Right Resource Fit project, leveraging the MERN stack.

```
hire-helper/
 ├── backend/
 │   ├── config/
 │   ├── middleware/
 │   ├── models/
```

```
│   ├── routes/
│   ├── uploads/
│   ├── utils/
│   ├── .env
│   ├── package.json
│   ├── package-lock.json
│   └── server.js
│
├── frontend/
│   ├── dist/
│   ├── node_modules/
│   ├── public/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── App.jsx
│   │   ├── index.css
│   │   ├── main.jsx
│   │   └── settings.css
│   │
│   ├── .env
│   ├── .gitignore
│   ├── eslint.config.js
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   ├── README.md
│   └── vite.config.js
│
```

└── README.md

Key Features

- **User Authentication with OTP:** Secure login and signup using OTP verification to ensure safe access for users.
- **Task Posting and Management:** Users can create tasks by providing details such as title, category, location, date, and description.
- **Service Provider Interaction:** Skilled workers can browse available tasks and offer their help based on their expertise.
- **Request Handling System:** Users can view incoming requests and accept or reject them based on preferences.
- **Real-Time Task Tracking:** Users can track the status of tasks such as pending, accepted, or rejected through a dashboard.
- **User Profile Management:** Allows users to update personal details, upload profile images, and manage account settings.
- **Notification System:** Provides alerts for task updates, request responses, and important activities.
- **Dashboard Interface:** A personalized dashboard displaying task summaries, request statuses, and quick actions.
- **Responsive Design:** The application is optimized for both desktop and mobile devices for a smooth user experience.
- **Secure Backend Integration:** Ensures safe data handling using technologies like Node.js, Express, and MongoDB.

Project Components

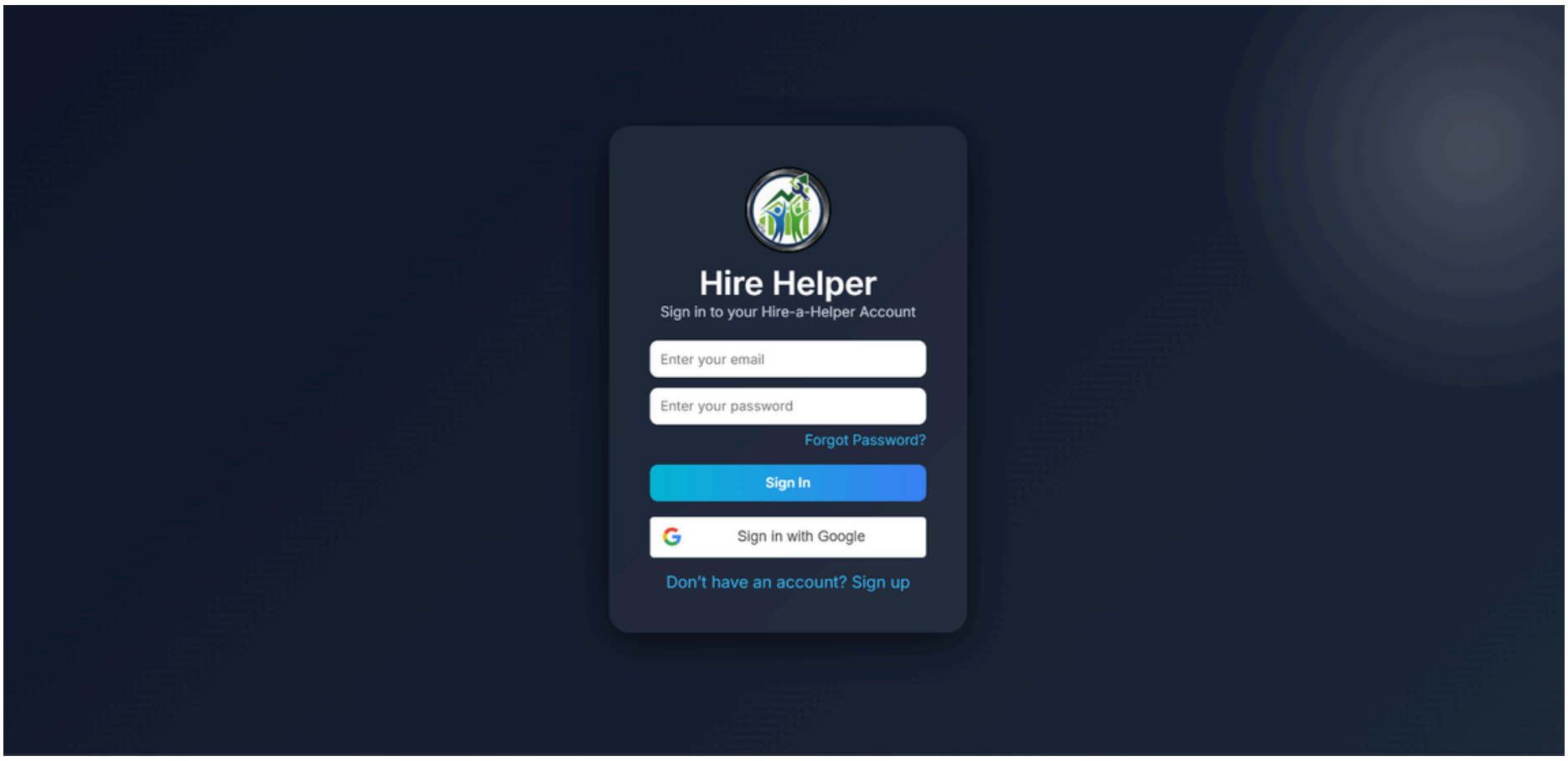
Here are the list of key components in the Right Resource Fit project, along with brief description of their functfunctionality.

Login Component

Description : Manages the user login process, handling inputs for email and password.

Key Features :

- **Password Visibility Toggle** : Allows users to show/hide their password for easy input
- **'Remember Me' Functionality** : Saves user login credentials (using cookies or localStorage) for subsequent visits without needing to re-enter credentials.
- **From Validation** : Ensures that the user provides valid credentials before submission.
- **Error Handling** : Displays error messages in case of incorrect credentials.

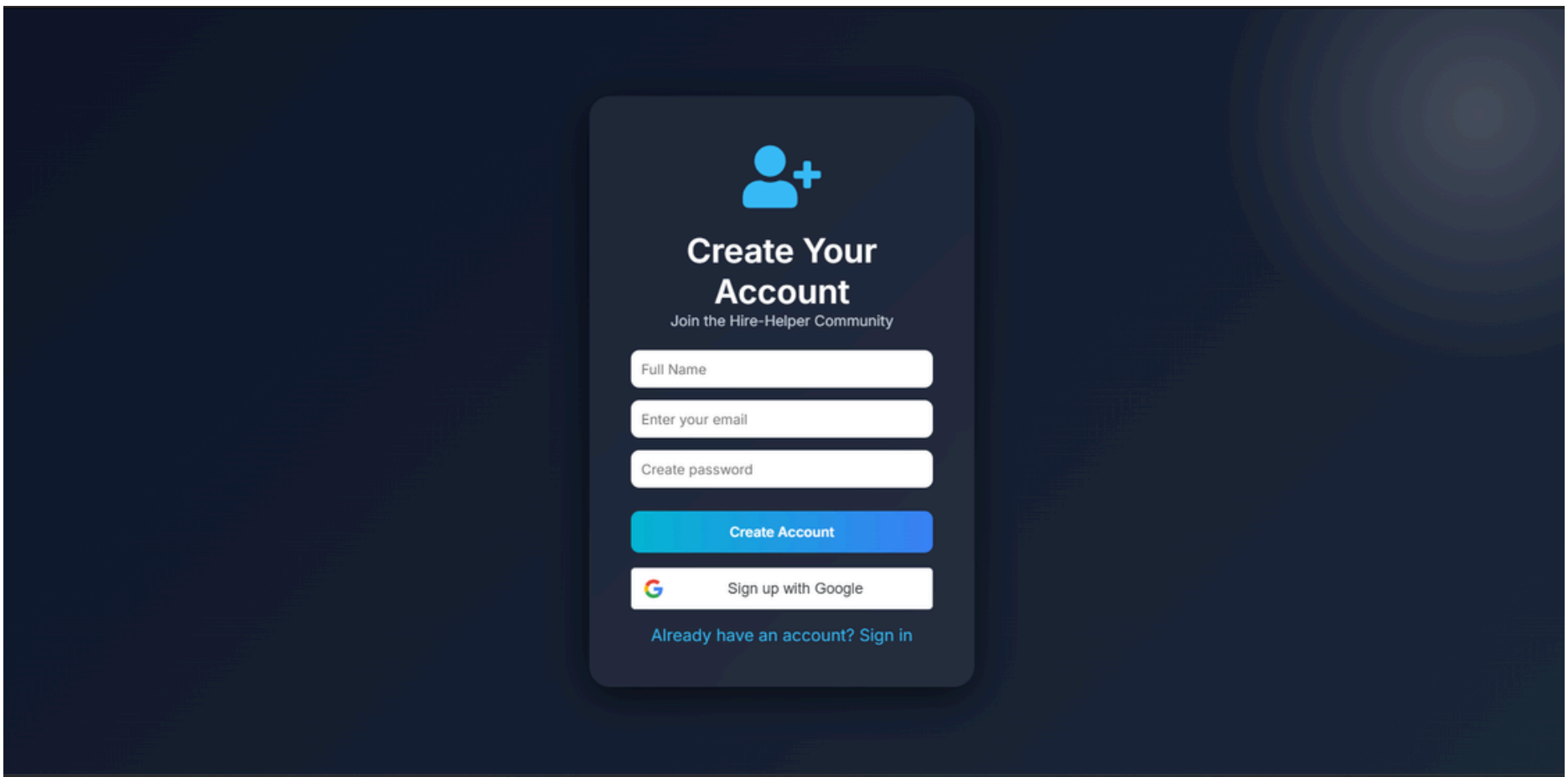


Create Account Component

Provides signup functionality for both job seekers (freelancers) and businesses (companies).

Key Features :

- **User Role Selection**: Allows users to select whether they are a job seeker or a company.
- **Form Fields**: Includes fields for email, password, name, and additional details (e.g., portfolio for job seekers, company name for businesses).
- **Validation**: Ensures that all necessary fields are filled out correctly and securely.
- **Submit & Success Handling**: On successful registration, redirects users to their appropriate dashboard (job seeker or business).

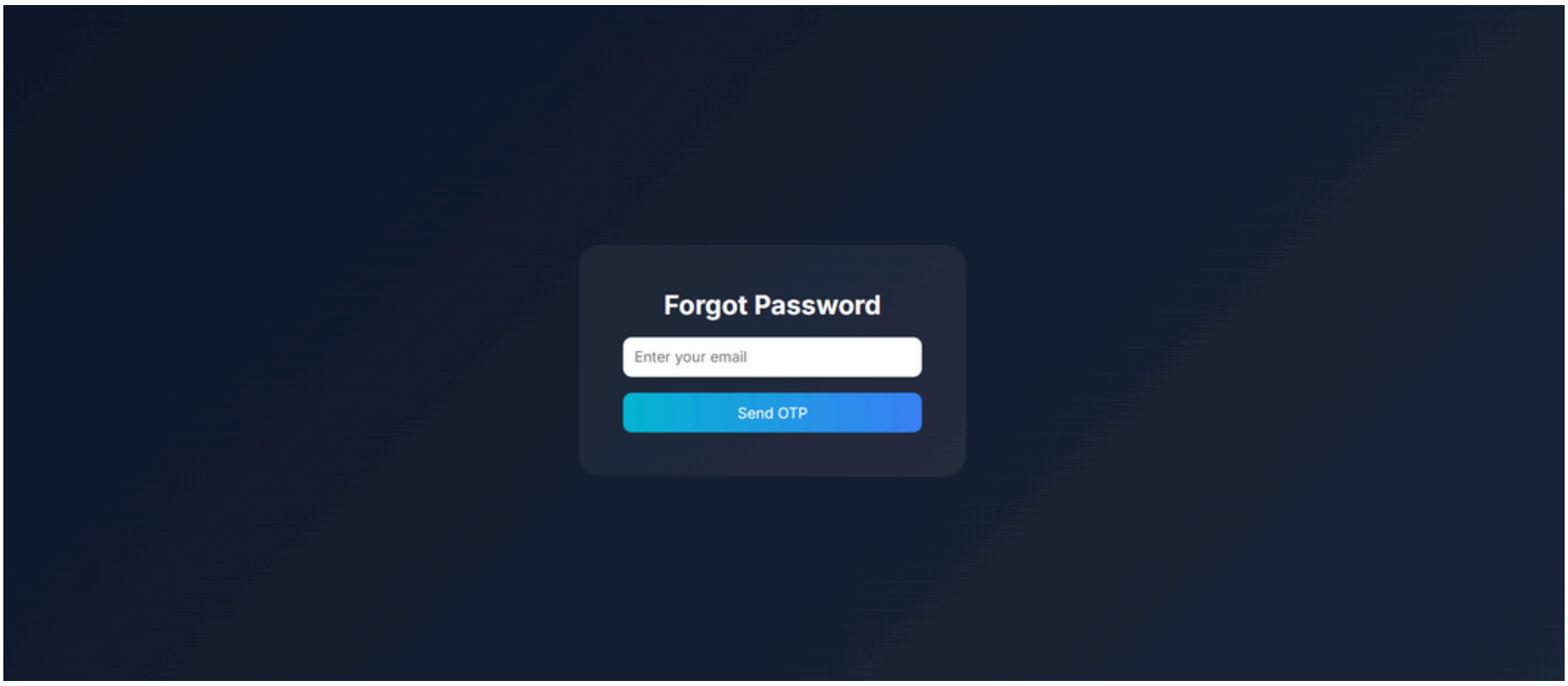


Forgot Password:

Description : Allows users to reset their password securely in case they forget their login credentials by using OTP verification.

Key Features :

- **Password Reset Request:** Users can request a password reset by entering their registered email
- **OTP Generation:** A one-time password (OTP) is sent to the user's email for verification
- **OTP Validation:** Users must enter the correct OTP to proceed
- **New Password Setup:** After verification, users can create a new password
- **Security Measures:** OTP is valid for a limited time and can be used only once
- **Error Handling:** Displays messages for invalid email, incorrect OTP, or expired OTP

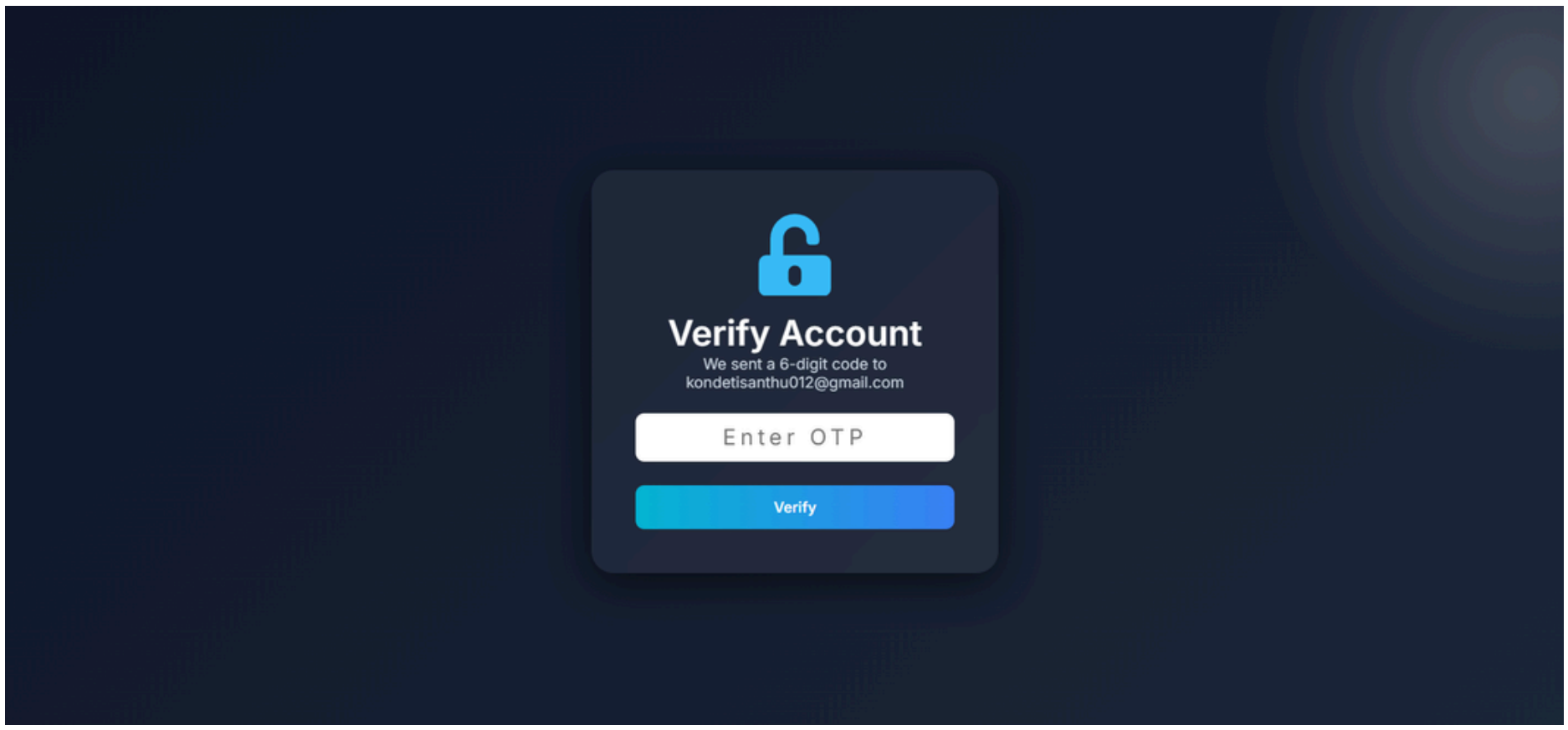


OTP Component :

Description : The OTP Verification component is used to securely authenticate users during signup, login, or password reset by validating a one-time password sent to the user's registered email or mobile number.

Key Features :

- **OTP Generation:** Generates a unique one-time password when the user initiates authentication or password reset
- **OTP Delivery:** Sends the OTP to the user's registered email (or phone) using a secure service (e.g., Firebase)
- **OTP Input Field:** Provides an input interface for users to enter the received OTP
- **OTP Validation:** Verifies the entered OTP with the generated one to authenticate the user
- **Time-Limited Validity:** OTP is valid only for a short duration to enhance security
- **Resend OTP Option:** Allows users to request a new OTP if they did not receive or if it expires
- **Error Handling:** Displays appropriate messages for incorrect, expired, or invalid OTP



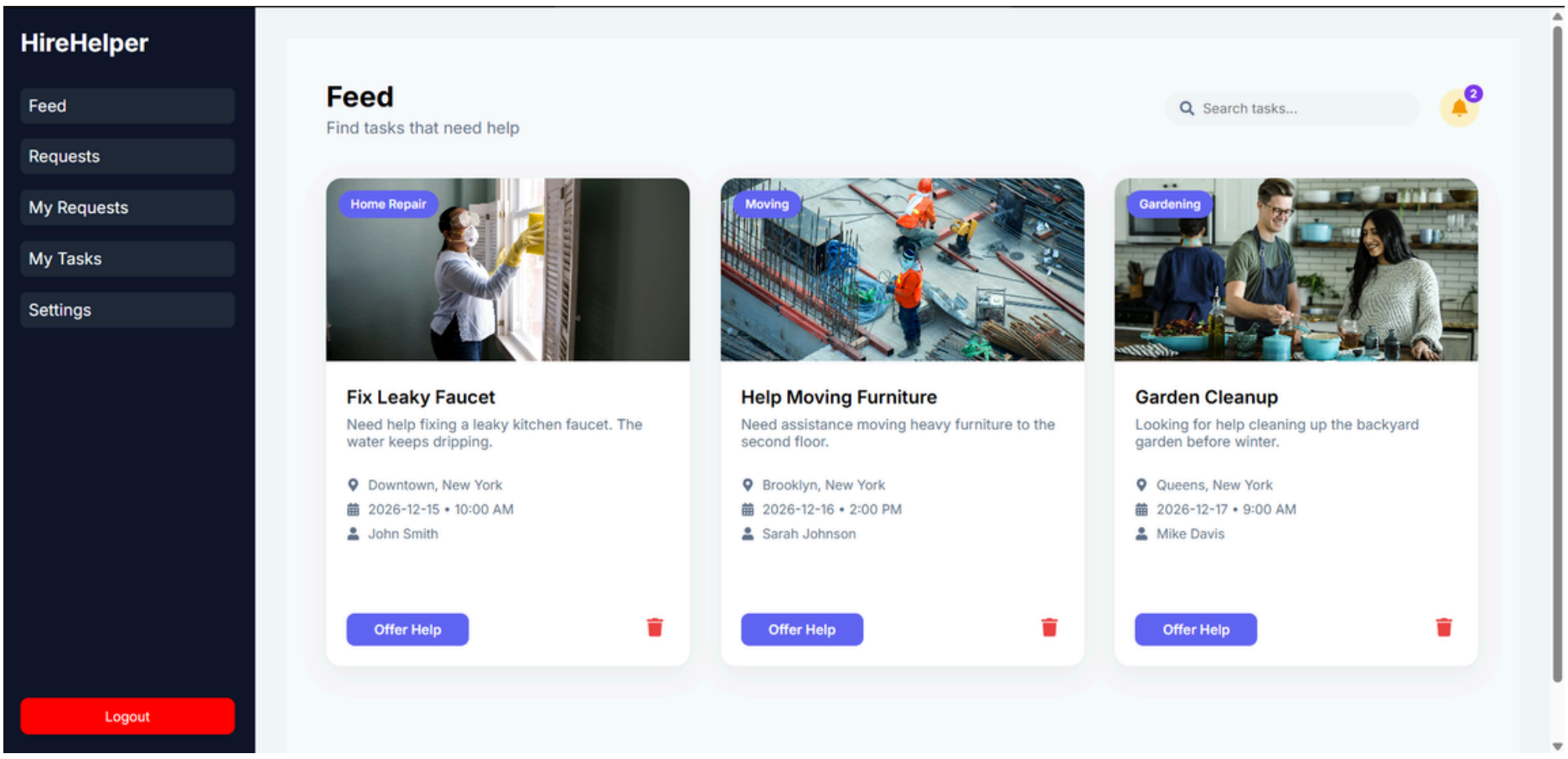
Feed Component :

Description: The Feed Page component displays a list of available tasks posted by users, allowing service providers to browse and select tasks based on their skills, location, and availability.

Key Features :

- **Task Listing:** Displays all available tasks with details such as title, description, location, date, and category
- **Card-Based UI:** Each task is presented in a structured card format for easy readability
- **Offer Help Option:** Allows service providers to respond to tasks by offering their help
- **Real-Time Updates:** Newly posted tasks are updated dynamically in the feed

- **Search and Filtering:** Enables users to filter tasks based on category, location, or type
- **Responsive Design:** Optimized for both desktop and mobile viewing
- **User-Friendly Interface:** Clean layout for smooth browsing experience

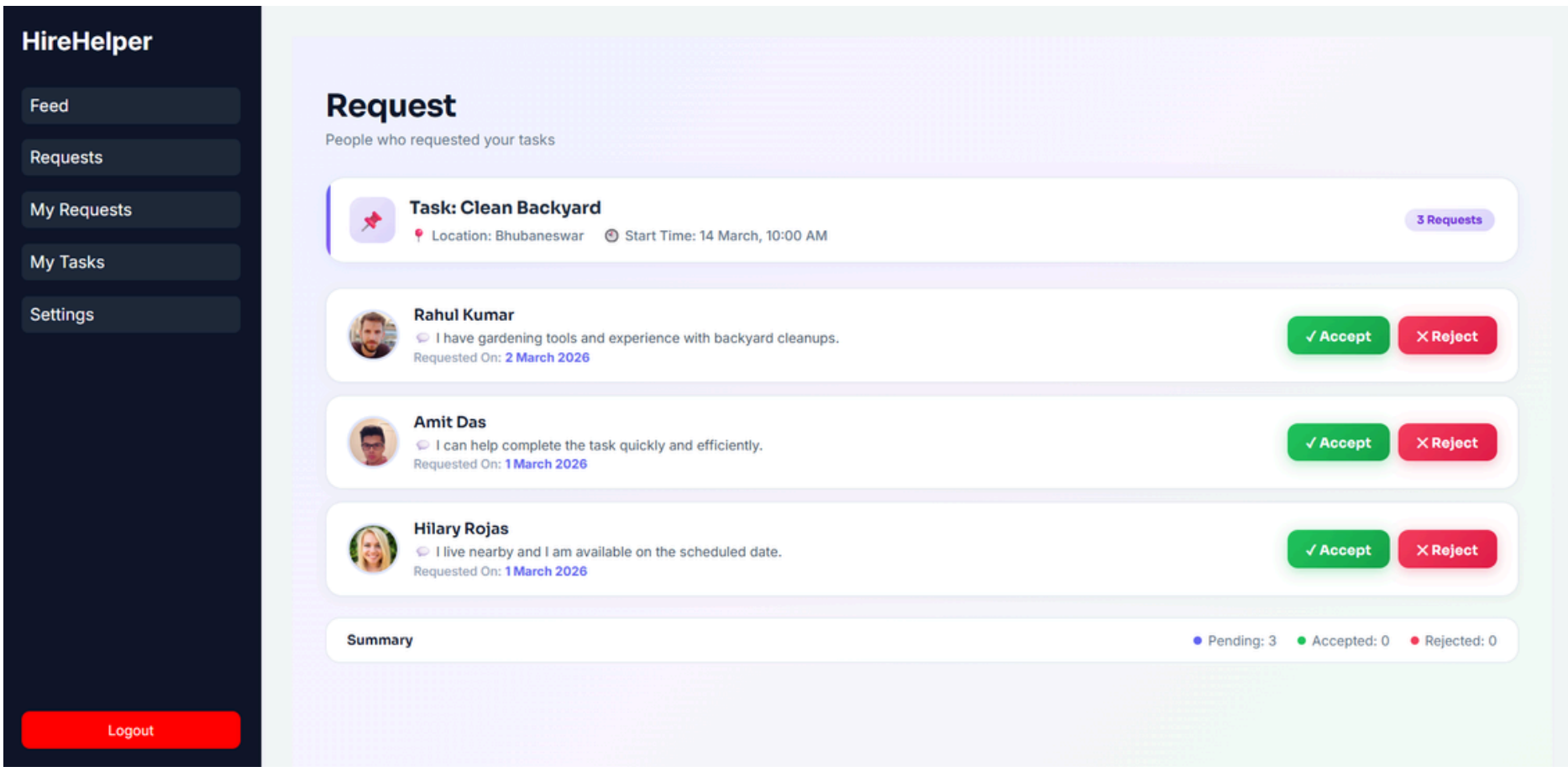


Request Component:

Description:The Requests component allows users to view and manage all incoming requests for the tasks they have posted, enabling them to accept or reject service providers based on their preferences.

Key Features:

- **Request Listing:** Displays all incoming requests from service providers for a specific task
- **User Details View:** Shows details of applicants such as name, contact, and basic information
- **Accept/Reject Actions:** Allows users to accept or reject requests easily
- **Status Management:** Updates request status (Pending, Accepted, Rejected) in real-time
- **Multiple Applicants Handling:** Supports multiple requests for a single task
- **Clear UI Display:** Organized layout for easy decision-making
- **Instant Feedback:** Notifies service providers about acceptance or rejection

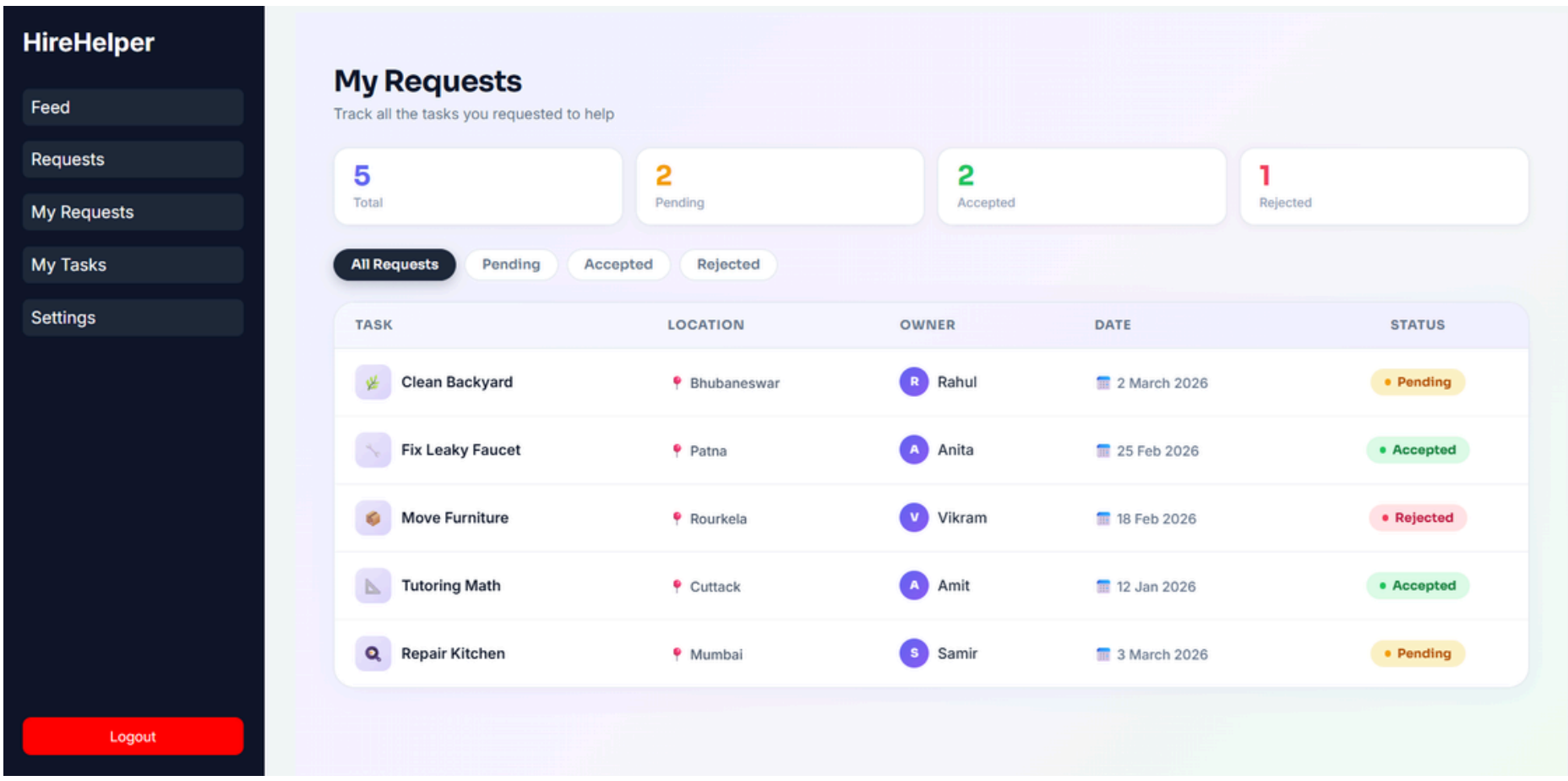


My Request Component:

Description:The My Requests component provides users with a centralized dashboard to view and track all the tasks they have posted or responded to, along with their current status.

Key Features:

- **Task Overview:** Displays a list of all user-related tasks in one place
- **Status Tracking:** Shows the current status of each request (Pending, Accepted, Rejected)
- **Organized Layout:** Presents data in a structured format for easy understanding
- **Quick Access:** Allows users to quickly view details of each task or request
- **Real-Time Updates:** Automatically updates task statuses as actions are performed
- **Filtering Options:** Users can filter requests based on status or category
- **User-Friendly Interface:** Clean and simple design for better usability

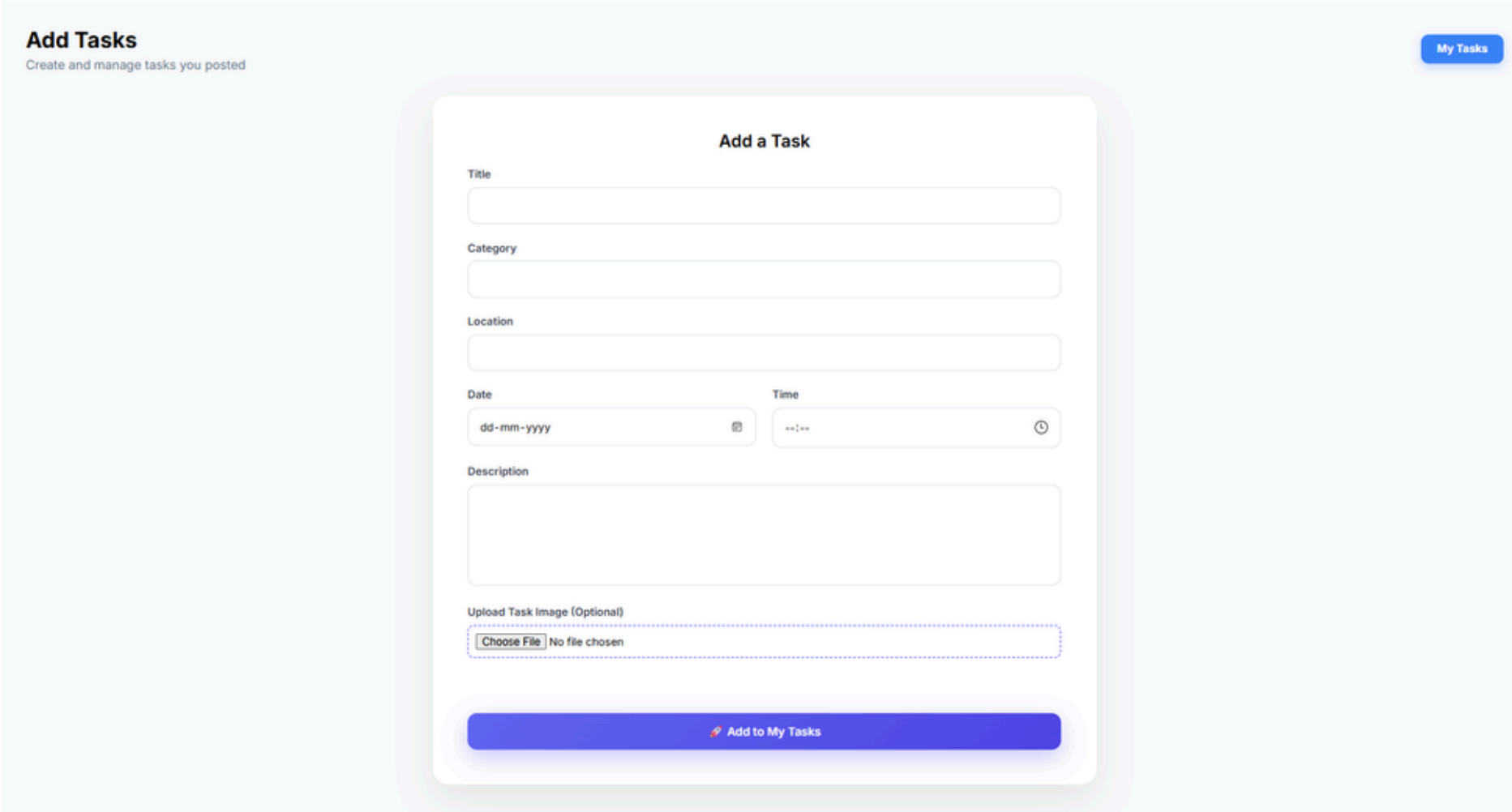


<u>My Task Component:</u>

Description :The My Task component allows users to view and manage all the tasks they have created on the platform, providing a clear overview of task details and their current progress.

<u>Key Features :</u>

- **Task Listing:** Displays all tasks posted by the user with relevant details such as title, category, location, date, and description
- **Status Tracking:** Shows the current status of each task (Open, In Progress, Completed, Closed)
- **Edit Task Option:** Allows users to update task details if needed
- **Delete Task Option:** Enables users to remove tasks that are no longer required
- **View Requests:** Provides access to view incoming requests for each task
- **Real-Time Updates:** Reflects changes in task status instantly
- **Organized Interface:** Clean layout for easy management of multiple tasks

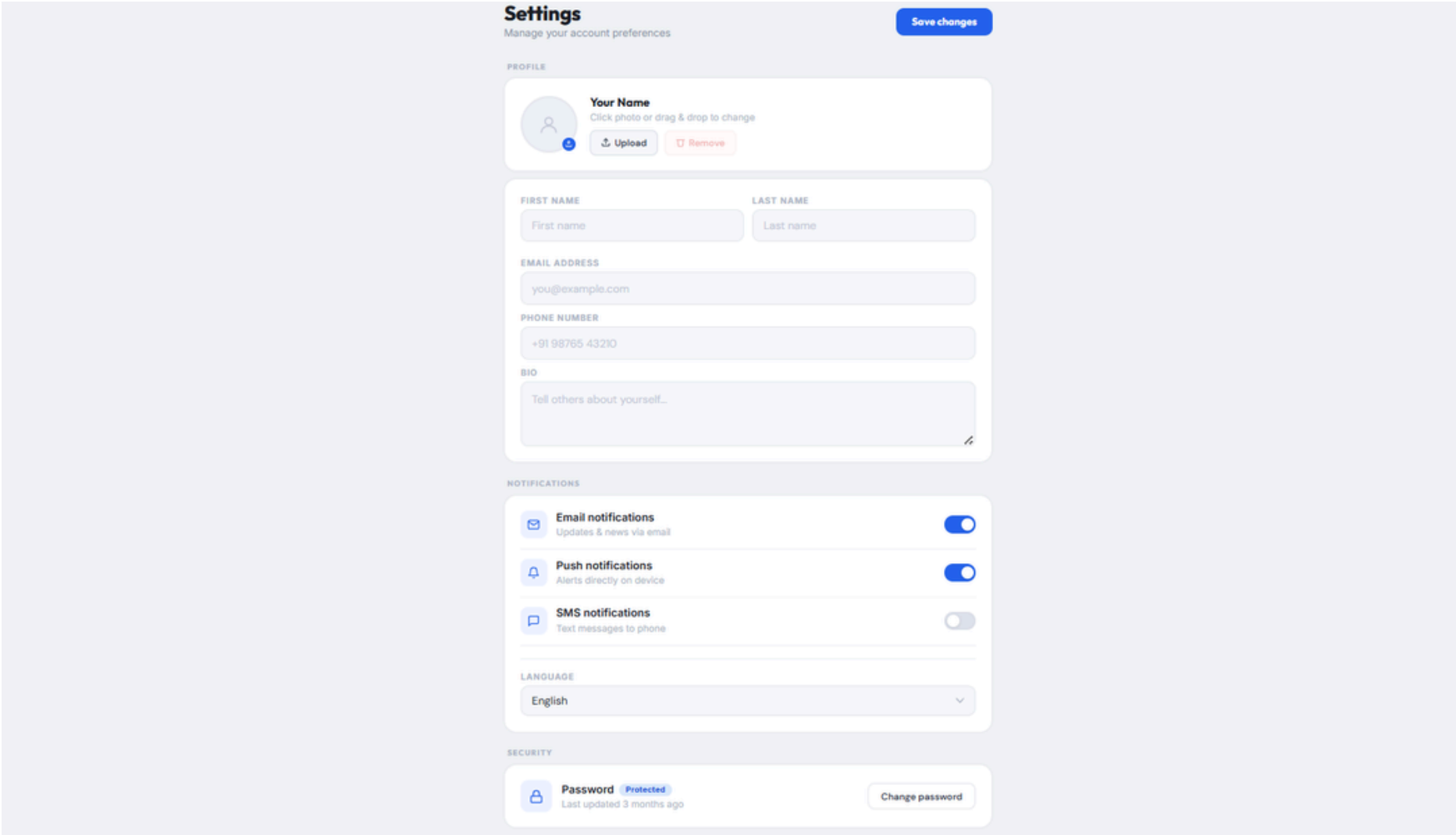


Settings Component:

Description :The Settings component allows users to manage and update their account preferences, personal information, and security settings, providing full control over their profile and application experience.

Key Features :

- **Profile Management:** Users can update personal details such as name, email, phone number, and profile picture
- **Account Settings:** Allows modification of basic account preferences
- **Password Management:** Enables users to change or update their password securely
- **Notification Settings:** Users can enable or disable notifications based on their preferences
- **Security Options:** Provides options to manage account security and privacy settings
- **User-Friendly Interface:** Clean and simple layout for easy navigation
- **Data Persistence:** Ensures all updated settings are saved and reflected immediately



API Components

User Authentication & Management

User Registration

Route: POST http://localhost:5173/signup.

Description: Register a new user account.

Input:

{

"username": Santu",

"email": "Santu@gmail.com",

"password": "Santu@123”

}

Output:

- 200 OK: { "message": "Signup successful" }
- 400 Bad Request: { "error": "Invalid input" }

User Login

Route: POST http://localhost:5173/login

Description: Authenticate a user.

Input:

```
{
    "email": "Santu@gmail.com",
    "password": "Santu@123”
}
```

Output:

- 200 OK: { "message": "Login successful"}
- 401 Unauthorized: { "error": "Invalid credentials" }

Password Reset Request

- **Route**: POST http://localhost:5173/send-otp
- **Description**: Send OTP for password reset.

Input:

```
{
    "email": "Santu@gmail.com"
}
```

Output:

- 200 OK: { "message": "OTP sent successfully" }
- 400 Bad Request: { "error": "Failed to send OTP" }

User Logout

- **Route**: POST http://localhost:1000/logout
- **Description**: Logout a user.

Input: None

Output:

- 200 OK: { "message": "Logged out successfully" }

Applications

Get All Requests for a Task

- **Route**:
  GET http://localhost:5173/requests/:taskId
- **Description**:
  This route retrieves all requests submitted by service providers for a specific task. Each request includes user details such as name, message, request date, and current status. These requests are displayed in the Requests page, where the task owner can review and take action (accept/reject).

Input:

taskId: ID of the task

Output:

- 200 OK: [
  {
  "requestId": "1",
  "userName": "Rahul Kumar",
  "message": "I have gardening tools and experience with backyard cleanups.",
  "requestedDate": "2026-03-02",
  "status": "Pending"

  }]

Accept Request

- **Route:** PUT http://localhost:5173/requests/:id/accept
- **Description:**This route allows the task owner to accept a request from a service provider. Once accepted, the request status is updated and the selected user is assigned to the task.

Input:
id: Request ID

Output:

- 200 OK: { "message": "Request accepted successfully" }
- 400 Bad Request: { "error": "Unable to accept request" }

Reject Request

- **Route:**PUT http://localhost:5173/requests/:id/reject
- **Description:** This route allows the task owner to reject a request. The request status is updated accordingly and the service provider is notified.

Input:
id: Request ID

Output:

- 200 OK: { "message": "Request rejected successfully" }
- 400 Bad Request: { "error": "Unable to reject request" }

Request Summary

- **Route:**GET http://localhost:5173/requests/:taskId/summary
- **Description:** This route provides a summary of all requests for a task, including the number of pending, accepted, and rejected requests. It helps users quickly understand the overall status of their task.

Input:
taskId: ID of the task

Output:

- 200 OK: { "pending": 3,"accepted": 0,"rejected": 0}

Create Task

- **Route:** POST http://localhost:5173/tasks
- **Description:** Create a new task with details like title, category, location, date, time, and description.

Input:

```
{
    "title": "Fix Leaky Faucet",
     "category": "Home Repair",
     "location": "Hyderabad",
     "date": "2026-03-30",
     "time": "10:00 AM",
     "description": "Need help fixing a leaking faucet",
     "image": "<optional image file>"
}
```

Output:

- 200 OK: { "message": "Task created successfully" }
- 400 Bad Request: { "error": "Invalid task data" }

Get My Tasks

- **Route:** GET http://localhost:5173/tasks/my
- **Description:** Retrieve all tasks created by the logged-in user.

Input:
None

Output:

- 200 OK: [
  { "taskId": "1", "title": "Fix Leaky Faucet", "status": "Open" },
  ...
  ]

Delete Task

- **Route:** DELETE http://localhost:5173/tasks/:id
- **Description:** Delete a specific task.

Input:
id: 673ff1845ca9da30126554c8

Output:

- 200 OK: { "message": "Task deleted successfully" }
- 404 Not Found: { "error": "Task not found" }

Notifications

Get Notifications

- **Route:** GET http://localhost:5173/notifications/:userId
- **Description:** Retrieve all notifications for a specific user such as request updates, task status changes, and alerts.

Input:
userId: 671f1b74f350e165d3958f65

Output:

- 200 OK: [

{ "notificationId": "1", "message": "Your task request has been accepted", "status": "Unread" ..... },]

Mark Notification as Read

- **Route:** PUT http://localhost:5173/notifications/:id/read
- **Description:** Mark a specific notification as read.

Input:
id: 67436b52d9cbbf62f5a7b9ec

Output:

- 200 OK: { "message": "Notification marked as read" }
- 400 Bad Request: { "error": "Unable to update notification" }

Delete Notification

- **Route:** DELETE http://localhost:5173/notifications/:id
- **Description:** Delete a specific notification.

Input:
id: 67436b52d9cbbf62f5a7b9ec

Output:

- 200 OK: { "message": "Notification deleted successfully" }
- 404 Not Found: { "error": "Notification not found" }

Error Handling and Troubleshooting

Effective error handling and troubleshooting ensure the smooth operation of the Right Resource Fit platform. Below are some common errors that users and developers may encounter, along with steps for troubleshooting and resolving them.

Gmail SMTP Not Working

Error Description: The platform uses SMTP to send email notifications (e.g., OTPs,

confirmation emails). If SMTP fails, emails won't be sent

Troubleshooting Steps:

Check Gmail Settings: Ensure that less secure app access is enabled in your Gmail account. Gmail may block connections from apps it considers less secure.

- Navigate to your Google Account settings.
- Under "Less secure app access," turn on access.

Check Credentials:

Double-check the email and password provided in the configuration to ensure they are correct.

- Check SMTP Settings:
- SMTP Server: smtp.gmail.com
- Port: 587 (for TLS)
- Ensure the secure: true flag is set in your SMTP configuration.
- Check 2-Step Verification: If you have 2-step verification enabled, you need to generate and use an App Password instead of your regular Gmail password.

Port Conflicts

Error Description: The application may fail to start due to port conflicts (e.g., the server tries to bind to port 3000, but it's already being used by another application).

Troubleshooting Steps:

Check Which Service is Using the Port:

On Linux/macOS, you can use the following command to check:

lsof -i :3000

On Windows, use:

netstat -ano | findstr :3000

Stop the Conflicting Process:

If you find a process using the port, you can stop it by identifying its PID and killing it.

On Linux/macOS, run:

kill -9 <PID>

On Windows, use:

taskkill /PID <PID> /F

Change the Port: If you can't stop the conflicting process or prefer to use a different port, update your application's configuration file to use a new port number (e.g., 3001 or 8080). Example (Node.js Express App):

Database Connection Failures

Error Description: The application may fail to connect to the database due to incorrect configuration or service

unavailability.

Troubleshooting Steps:

Check Database Credentials: Ensure that the database username, password, and connection string in the configuration are correct.

- For MongoDB, check the connection string format and the database's host URL.
- Ensure Database is Running: Make sure that the MongoDB or other database service is running on your local machine or remote server.
- Check Database Access Permissions: Ensure that the database user has sufficient permissions to perform operations (e.g., read, write).

API Request Failures

Error Description: API requests may fail due to misconfiguration or incorrect endpoint usage.

Troubleshooting Steps:

- **Check Endpoint URL**: Ensure that the correct API endpoint is being used. Check the route and URL in the backend and the client-side code.
- **Check API Authentication**: If your API requires authentication (e.g., API keys or tokens), ensure that these credentials are correctly included in the request headers.
- **Examine Response Codes:** Check the HTTP status codes returned by the API. Common errors include
- **404 (Not Found):** The requested endpoint is incorrect or missing.
- **500 (Internal Server Error):** There's an issue with the server; check logs for more details.
- **401 (Unauthorized):** Authentication issues, such as missing or invalid API keys.

File Upload Failures

Error Description: Users may encounter issues when uploading files (e.g., profile images, portfolio items).

Troubleshooting Steps:

- **Check File Size Limits:** Ensure that the file size does not exceed the server's upload limits. In Node.js, for example, you can increase the upload size in the body parser middleware
- **Verify File Type:** Ensure that only allowed file types (e.g., .jpg, .png, .pdf) are uploaded. Validate file types both on the client-side and server-side. Example (Node.js File Type Validation)
- **Check Disk Space:** Ensure the server has enough disk space to store the uploaded files.

Authentication Issues (JWT or Session Timeout)

Error Description: Users may experience issues logging in or maintaining sessions due to token expiration or invalid credentials.

Troubleshooting Steps:

- **Check Token Expiry**: If you are using JWT tokens, ensure that the token hasn't expired. JWT tokens typically have an expiration time encoded in them.
- **Ensure Correct Login Credentials:** Check that the user is providing the correct username and password and that your backend is validating these correctly.
- **Check Session Storage:** If using sessions, ensure the session cookie is correctly stored and sent with each subsequent request.

Frontend Rendering Issues

Error Description: The frontend UI may fail to load or display incorrectly due to errors in rendering or incorrect data passed to components.

Troubleshooting Steps:

- **Check JavaScript Errors:**Use the browser's developer tools (F12) to check for JavaScript errors in the console. Look for error messages related to missing variables, incorrect function calls, or syntax issues.
- **Inspect DOM Elements:** Ensure the correct data is being passed to your UI components. Use console.log() to log the data being passed into templates or components.
- **Ensure Dependencies Are Installed:** Check if all the required frontend dependencies are installed (e.g., React.js) Run npm install or yarn install to install any missing packages.

General Troubleshooting Tips

- **Clear Cache:** Sometimes, browser or application caches can cause issues. Clear the cache or try running the application in incognito mode.
- **Check Server Logs:** Always check the server logs for detailed error messages. They can provide more context on what's going wrong.
- **Use Debugging Tools:** Utilize debugging tools like Chrome DevTools, Visual Studio Code debugger, or backend-specific debuggers to step through the code and identify issues.

Maintenance and Support

Procedures for Regular Maintenance

Backing Up the Database

- Use MongoDB's built-in backup tools or cloud-based solutions like MongoDB Atlas to schedule regular backups.
- Store backups securely in a cloud storage service (e.g., AWS S3, Google Cloud Storage) to prevent data loss.

- Rotating API Keys or Credentials.

Regularly update sensitive credentials stored in environment variables (e.g., .env files).

- Use tools like AWS Secrets Manager or Azure Key Vault for secure storage.
- Monitoring Logs and Server Health
- Implement logging with tools like Winston or ELK Stack for application logs.
- Use monitoring tools like New Relic, Prometheus, or Datadog to track server performance and uptime.

How Users Can Report Issues or Request Support

- Provide a dedicated Support Email Address (e.g., support@rightresourcefit.com).
- Include a link to a Support Pagewhere users can:
- Submit issue reports.
- Access self-help guides or FAQs.
- Set up a ticketing system using tools like Zendesk or Jira for efficient issue tracking.

Version Control and Updates

Using Git for Version Control

Branching Strategies:

- Use the main branch for production-ready code.
- Create feature branches for new features or bug fixes.
- Use release branches for testing updates before merging into the main branch.

Commit Messages:

- Follow a standard format such as [feature]: Add user authentication or [fix]: Resolve login bug.
- Applying Updates and Migrations Safely
- Test updates in a staging environment before deploying to production.
- Perform database migrations using tools like Mongoose migration scripts.

Glossary or FAQ

Glossary

- **Matching Algorithm:** A system that matches candidates to jobs based on their skills, experience, and availability.
- **Skills Portfolio:** A section in a user profile where professionals showcase their expertise, certifications, and completed projects.
- **Availability Tracking:** A feature that allows businesses to view real-time availability of candidates.

FAQs

- Why is the Gmail SMTP not working?

Ensure that "Allow less secure apps" is enabled in Gmail settings or use OAuth 2.0 for secure access.

- What to do if I encounter a port conflict?

Use the command lsof -i :3000 to identify the service occupying the port and stop it.

- How can I reset my password?

Go to the login page and click on "Forgot Password." Follow the instructions to reset it via email.

Account and Profiles

- How can I update my profile details?

Log in to your account, navigate to the Profile section in the dashboard, and click Edit Profile to make updates.

- Why was my profile rejected?

Profiles are reviewed for accuracy and completeness. Ensure that all required fields are filled out and that your portfolio includes valid work examples.

Jobs and Applications

- How do I post a Job as a business?

Navigate to the Post a Project section, fill in the Job details (e.g., skills required, budget, and deadline), and submit it for review.

- Can I apply for multiple Jobs at the same time?

Yes, professionals can apply for multiple Jobs, but it's advised to prioritize projects that align closely with their skill set and availability.

- What happens if a project is canceled after it starts?

If a project is canceled, payments already made will follow the platform's refund or dispute resolution policy.

Technical Issues

- Why am I unable to upload my portfolio?

Ensure that your files meet the size and format requirements (e.g., PDF, PNG, JPG, under 10MB). If the issue persists, clear your browser cache and try again.

- I’m not receiving email notifications. What should I do?

Check your spam or junk folder. Add the platform’s email address to your safe sender list. If the problem continues, verify that your email address is correctly configured in your profile.

Future Enhancements

Integration with Third-Party Tools

Add integration with tools like Slack for notifications and Zapier for workflow automation.

AI-Driven Recommendations

Implement AI algorithms to recommend the best candidates or projects based on past trends.

Enhanced Reporting and Analytics

Provide detailed insights into hiring trends, project performance, and user engagement through an analytics dashboard.